

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №3

Выполнила:

Левандо Ксения

Группа:

J3211

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк [Vue.JS](#). По итогам приложение должно соответствовать следующим требованиям:

- Должен быть подключён роутер
- Должна быть реализована работа с внешним API (желательно посредством axios)
- Разумное деление на компоненты (продемонстрируйте понимание компонентного подхода)
- Использование composable для выделения повторяющегося функционала в отдельные файлы

Ход работы

1. Создадим проект

```
npm init vue@latest
```

Проделаем шаги аналогичные HW5 и получим начальный вид проекта.

- App.vue

```
<template>
  <div class="d-flex flex-column min-vh-100">
    <router-view />
  </div>
</template>

<style>
body {
  margin: 0;
}
</style>
```

- src/router/[index.js](#) (начальный вид)

```
import { createRouter, createWebHistory } from "vue-router";

import HomeView from "@/views/HomeView.vue";
```

```
const routes = [
  { path: "/", component: HomeView },
  { path: "/events", component: HomeView },
];

const router = createRouter({
  history: createWebHistory(),
  routes
});

export default router;
```

Отредактируем также index.html и подключим там бутстрап:

```
<!-- bootstrap css -->
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
rel="stylesheet"
integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
crossorigin="anonymous">
```

```
<!-- Add JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
integrity="sha384-FKyoEForCGlyvw9Hj09JcYn3nv7wiPV1z7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
crossorigin="anonymous"></script>
```

Далее будем переносить странички по одной:

2. Перенос HomeView.vue

Создадим файл такого вида:

```
<template>

<div>

  <!-- Navigation -->

  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">

    . . .
```

```

</nav>

<!-- Content -->

<main class="container my-5 flex-grow-1">
. . .

</main>

<!-- Footer -->

<footer class="site-footer mt-auto">
. . .

</footer>

</div>
</template>

```

И перенесем сюда HTML-разметку, адаптируя ее под Vue (замена href на `<router-link>`, обработка событий через `@click`, `@submit` и т.д.).

Например, ссылки по типу `<a href="events.html">` заменим на:

```
<router-link to="/events" class="nav-link"> Events </router-link>
```

И так далее

3. Аналогично переносим странички Events и Event

При переносе прокомментируем кнопку `toggleTheme`, к ее функционалу вернемся далее.

4. Продолжаем процесс поэтапного переноса страниц, в итоге получаем:

- Главная страница → HomeView.vue
- Страница событий → EventsView.vue
- Страница конкретного события → EventView.vue
- Страницы авторизации и регистрации → LoginView.vue, RegisterView.vue
- Дашборды пользователя и организатора → DashboardView.vue, OrganizerDashboardView.vue

=> попутно внесем небольшие правки по organizer dashboard:

- 1) Дата события не в прошлом (не раньше сегодняшней): валидация в форме
- 2) Дефолтная картинка для афиши события, если URL не указан

6. Перенос css

Перенесем в неизменном виде файлы themes.css и style.css в новую папку. Подключим их через index.html

```
<link rel="stylesheet" href="/assets/css/style.css">

<link rel="stylesheet" href="/assets/css/themes.css">
```

7. Настройка роутинга

Настроим vue-router:

- добавлены маршруты для всех страниц
- реализована навигация без перезагрузки страницы
- добавлены переходы после логина/логаута

8. Выделение components

Во избежание переиспользования кода вынесем общие части интерфейса:

- Navbar.vue — навигационная панель

- Footer.vue — футер
- AppModal.vue — универсальное модальное окно
- ConfirmModal.vue — модальное окно подтверждения

И подключим их к нужным views (а Navbar и Footer к App.vue). Это позволяет избежать дублирования кода на разных страницах.

9. Работа с API

Для взаимодействия с сервером подключён Axios:

- создан экземпляр api (instance.js) с базовым URL
- реализованы запросы для:
 - получения событий
 - создания и удаления событий
 - покупки и возврата билетов
 - авторизации пользователей и организаторов

10. Введение composables

Для улучшения структуры и переиспользования логики созданы composables:

- useAuth.js - отвечает за авторизацию:
 - хранение пользователя и организатора
 - состояния isAuth и isOrganizer
 - функции login, loginOrganizer, logout
- useTheme.js- реализует переключение темы:
 - хранение состояния темы
 - синхронизация с localStorage
 - изменение data-theme в DOM
- useModal.js: управление модальными окнами:
 - открытие/закрытие модалок
 - передача заголовка, текста и типа

11. Итоговый вид

В результате проект полностью перенесён на Vue:

- компонентная архитектура
- централизованная логика (composables)
- SPA-навигация через router
- работа с API через Axios

Вывод:

В рамках работы над ЛР-3 я научилась уже не только стартовать и создавать простейший проект на Vue, как в HW-5, а перенесла уже созданный ранее сайт, адаптировав его, организовав работу с API через Axios, разбив интерфейс на переиспользуемые компоненты (Navbar, Footer, модальные окна) и перенесла часть логики в composables.

Таким образом, в ходе выполнения задания я получила практические навыки разработки SPA-приложений на Vue, организации архитектуры проекта и управления состоянием приложения.